

DMEI

Dynamic Masking Embedded Indexing

Ephemeral-Entropy Indexing for OS Dispatch Tables:

Inline Tamper Detection and Index Obfuscation Against Write-Only Adversaries at $O(1)$ Cost

Eyal Braun

Independent Researcher · 2025

Abstract

We study integrity protection for command dispatch tables used in operating system kernels, interpreters, and embedded runtimes. These structures map commands or identifiers to function handlers and are critical to correct execution — corruption redirects control flow and compromises system integrity.

We introduce ephemeral-entropy indexing: a construction principle in which storage positions depend on a value destroyed during construction. DMEI (Dynamic Masking Embedded Indexing) instantiates this principle as a cuckoo hash table providing: (1) inline tamper detection at probability $1-2^{-32}$, no MAC; (2) index obfuscation — a write-only adversary cannot locate a specific entry without a read primitive; (3) $O(1)$ worst-case lookup; (4) session binding at probability $1-2^{-32}$.

DMEI targets OS command dispatch tables — kernel syscall tables, interpreter opcode tables, and embedded handler routers — where Rowhammer and limited write primitives are documented attack classes and lookup must complete in under 10 ns. DMEI achieves all four properties at 5.5 ns median: $1.53\times$ a plain cuckoo table and $8\times$ faster than MAC-authenticated alternatives.

Keywords: *ephemeral-entropy indexing, OS command dispatch, Rowhammer, tamper detection, cuckoo hashing, index obfuscation, write-only adversary, kernel security, control-flow integrity*

1. Introduction

1.1 A New Principle: Ephemeral-Entropy Indexing

Consider a data structure whose entries are stored at positions that depend on a value A that exists only during construction and is destroyed the moment construction ends. An adversary wanting to corrupt entry X must know where X is stored. Without a memory-read primitive to

recover A from the structure, the adversary faces a uniform distribution over all possible positions — targeted corruption requires first escalating to a read+write attack.

We call this ephemeral-entropy indexing: using a transient, irretrievable value as the randomness source for storage placement. DMEI is its first instantiation as a hash table, motivated by a concrete and documented security problem in OS kernels. Ephemeral-entropy indexing combines naturally with inline integrity checking: every lookup must traverse the pointer table to retrieve the ephemeral-derived index, so the integrity check rides on that mandatory path at no extra cost.

1.2 The Problem: Command Dispatch Tables Under Write-Only Attack

Command dispatch tables are ubiquitous in system software. They map commands or identifiers to handler functions and are critical to correct execution. Three concrete instantiations motivate this work:

- **The Linux x86-64 syscall table maps 448 syscall numbers to `sys_*` handler function pointers. It remains writable after boot to support loadable kernel modules.** Kernel syscall tables.
- **Language runtimes (CPython, LuaJIT, SpiderMonkey) maintain opcode-to-handler dispatch tables built at interpreter startup and accessed on every bytecode instruction.** Interpreter opcode tables.
- **Real-time operating systems (FreeRTOS, Zephyr) and DO-178C-certified flight controllers route command identifiers to interrupt service handlers. Dispatch is on the critical path of every task switch.** Embedded command routers.

The attacker's goal: corrupt a single dispatch table entry to redirect control flow to attacker-controlled code, without triggering detection, and without a memory-read primitive. An attacker with a write-only primitive — Rowhammer, DMA write, or limited heap overflow — can compute the target address in a standard hash table and flip bits there. DMEI removes this capability.

1.3 Why Existing Defenses Are Insufficient

- `__ro_after_init` — inapplicable to tables modified after boot (Linux syscall table, plugin dispatch).
- LKRG — periodic checks. An attacker exploiting between check intervals is unaffected; DMEI detects at lookup time with zero interval.
- kCFI/FineIBT — catches exploitation of a corrupted pointer, not the corruption itself. A pointer redirected to a valid CFI target (a less-privileged handler) bypasses CFI entirely.
- ECC RAM — absent from most consumer and embedded hardware; multi-bit Rowhammer bypasses single-bit ECC.
- HMAC per slot — 100% tamper detection, but ~45 ns per lookup, exceeding the <10 ns dispatch budget.

Unlike CFI mechanisms that protect indirect branches at exploitation time, DMEI protects dispatch table integrity within the data structure — detecting corruption before any corrupted pointer is dereferenced.

1.4 Contributions

- Ephemeral-entropy indexing — a named, general construction principle for write-only-resistant data structures.
- DMEI — a fully correct cuckoo hash table providing all four security properties at $O(1)$ cost.
- Formal security theorems — Theorems 1–7 with complete proofs (Section 7).
- Honest analysis — explicit limits on scope: pointer table slot determinism, read+write adversary boundary, ASLR dependency.
- Verified implementation — correct cuckoo insertion, compiler memory barrier, data collision detection, 100% correctness at design operating point.

2. Background

2.1 Dispatch Table Sizes in Real Systems

A key claim of this paper is that the design operating point $n \leq 500$ keys is not an arbitrary limitation — it precisely covers the natural size of dispatch tables in every target application. The following table enumerates real dispatch table sizes from production systems:

System	Table Type	Entry Count	Reference
Linux x86-64 kernel	syscall table	448 entries	arch/x86/entry/syscalls/syscall_64.tbl
CPython 3.12	opcode dispatch	~119 opcodes	Python/ceval.c computed_gotos
LuaJIT 2.1	bytecode dispatch	97 opcodes	lj_vm.dasc BC_* definitions
seL4 microkernel	capability invocations	~50–200 caps	seL4 Reference Manual §3
FreeRTOS (typical)	ISR command table	16–64 handlers	Application-defined
DO-178C flight ctrl.	command router	32–128 handlers	Avionics domain practice
DMEI design range	all of above	$n \leq 500$	covers every row above

Table 1 — Real dispatch table sizes. Every target system falls within DMEI's $n \leq 500$ design range. For larger tables, $M=64N$ handles $n \leq 2000$ at the same rebuild probability.

The $n \leq 500$ constraint arises from the data collision probability formula $E[\text{coll}] = n^2/(2M)$. At $M = 16N = 1,048,576$ and $n = 500$: $E[\text{coll}] = 0.119$, $P(\text{rebuild}) = 11.2\%$. For tables larger than 500 entries — outside the scope of any known kernel dispatch structure — setting $M = 64N$ reduces rebuild probability to under 2% at $n = 2000$.

2.2 Rowhammer and the Write-Without-Read Property

Kim et al. [2014] showed DDR3 DRAM suffers disturbance errors: hammering two aggressor rows induces bit flips in adjacent victim rows that are never directly accessed. This is not a modeling assumption — it is a physical property of DRAM cell coupling. The Rowhammer attack is write-without-read by construction: the victim row is modified precisely because the attacker does not access it.

Seaborn and Dullien [2015] achieved full kernel privilege escalation from user space (CVE-2015-0565) by locating target PTEs via timing — feasible because the target was at a deterministically computable address. DRAMMER [van der Veen 2016] replicated this on ARM. TRRespass [Frigo 2020] demonstrated Rowhammer on DDR4. CVE-2016-2431 and CVE-2021-26708 document software write-without-read primitives in kernel code.

If the storage address is not computable without a read primitive, the attacker is reduced to blind hammering: random, unreliable, and far more detectable. This is the security benefit DMEI exploits.

2.3 Syscall Dispatch Performance Budget

The Linux x86-64 syscall entry path takes ~100–200 ns end-to-end. Dispatch table lookup must complete in under 10 ns. All benchmarks are microbenchmarks on synthetic key sets; end-to-end kernel module measurement is future work (Section 13). DMEI at 5.5 ns is within budget; HMAC at ~45 ns is not.

3. Threat Model

The central concern about this work's threat model is: why would an attacker lack a read primitive? In most memory exploits, read and write capabilities arrive together. This section answers that question precisely, by grounding each adversary model in documented attack classes where the read/write asymmetry is a physical or architectural property — not an assumption.

3.1 Model A — Fault Adversary (Read-Free by Physics)

The fault adversary induces memory corruption through hardware faults. The defining characteristic is that the corruption mechanism physically bypasses the CPU's memory access path:

- **Disturbance errors are induced by hammering adjacent DRAM rows. The victim row is flipped without being accessed — the attacker never issues a load to the victim address. This is the canonical write-without-read primitive.**Rowhammer.
- **Single-event upsets flip bits without any attacker-issued memory access. Relevant for DO-178C safety-critical avionics contexts.**Cosmic rays and alpha particles.
- **A malicious or buggy PCIe device issues DMA writes to a physical memory range. PCIe DMA is typically unidirectional in attacker-controlled malware — the attacker controls the DMA write engine but cannot issue arbitrary DMA reads into kernel virtual address space without additional exploit steps.**DMA corruption.

In Model A, the adversary has zero read capability. This is not an assumption — it is a consequence of the attack mechanism. Rowhammer in particular is write-without-read by construction. DMEI provides full protection: index locations are unguessable (Theorem 3) and any flip is detected (Theorem 2).

3.2 Model B — Limited Write Attacker (Read-Restricted by Architecture)

The limited write attacker exploits a software vulnerability that provides write access to a memory region but not arbitrary read access. Common examples:

- **A fixed-size overflow corrupts a contiguous memory region. The attacker controls write offsets but typically cannot read the overflowed region without a separate information leak.**Heap overflow.
- **A dangling pointer write allows one or more arbitrary writes. Without a corresponding use-after-free read, no information leak exists.**Write-after-free.
- **A driver exposes a write-only ioctl that writes to an arbitrary kernel address. No corresponding read ioctl exists.**Buggy kernel driver.

In Model B, the attacker can compute the deterministic pointer_table slot address (it depends only on the key s and session parameter P , which are known or guessable). The attacker can corrupt that slot, causing safe-failure NOT_FOUND — denial of service, not silent redirect. To silently redirect, the attacker needs the data_table index I , which requires reading ARM_A from the pointer_table. Without a read primitive, I is uniform over 2^{20} slots.

The question 'why can't the attacker also read?' is answered by the exploit class: heap overflows without an info-leak side channel are extremely common in the CVE database precisely because read and write primitives require separate vulnerability components. Model B covers the large class of write-only exploitation primitives where the info-leak component has not yet been found or chained.

3.3 Model C — Read+Write Adversary (Scope Boundary)

The full read+write adversary can both read ARM_A from the pointer_table and write a forged entry to the data_table. Under Model C, DMEI's index obfuscation is defeated. This is the honest scope boundary.

DMEI's security claim is precisely: it raises the required adversarial capability from write-only (Models A/B) to read+write (Model C). This is a meaningful security upgrade — it forces the attacker to chain two distinct exploit components (info-leak + write primitive) instead of one.

Tamper detection (Theorem 2) and session binding (Theorem 4) remain effective under Model C for adversaries who corrupt the data_table without first reading it — for example, a Rowhammer campaign targeting a physical address region that happens to overlap both tables.

3.4 Formal Adversary Definition

Let A be a probabilistic polynomial-time adversary that may issue write operations to memory cells but cannot read the ephemeral value A used during BUILD (Models A and B). Define the forgery advantage:

```
Adv_forge(A) = Pr[A produces (slot, data_table_entry) that passes LOOKUP  
without having read ARM_A]
```

Theorem 3 (Section 7) establishes $\text{Adv_forge}(A) \leq 1/M = 2^{-20}$ for index forgery. Theorem 2 establishes that any modification to a valid entry is detected with probability $\geq 1 - 2^{-32}$. Both bounds are tight — the adversary cannot improve beyond uniform guessing without a read primitive.

4. Related Work

4.1 Control-Flow Integrity

CFI [Abadi et al. 2005] restricts indirect branch targets to valid function entry points, preventing exploitation of a corrupted function pointer. LLVM CFI [Tice et al. 2014] and FineIBT implement CFI via compiler instrumentation and hardware features (ENDBR). These mechanisms operate at exploitation time — they catch what happens after a pointer has been corrupted. A dispatch entry corrupted to point to a valid CFI target that is merely less-privileged bypasses CFI while achieving the attacker's goal.

DMEI operates at a different point in the attack chain: it detects the corruption itself, before any corrupted pointer is dereferenced. The two mechanisms are complementary and can be deployed simultaneously.

4.2 Pointer Authentication (ARM PAC)

ARM Pointer Authentication [ARMv8.3, 2017] embeds cryptographic MACs in the high bits of 64-bit pointers using PACIA/AUTIA instructions. PAC protects pointer integrity at the hardware level, prevents forged pointers from being used, and requires no software MAC computation at the cost of $\sim 1\text{--}2$ ns per pointer use on modern ARMv8.3+ silicon.

PAC protects the pointer values stored in a dispatch table but does not protect the table index — an attacker who can locate and overwrite a PAC-signed pointer with another valid PAC-signed pointer (e.g., by swapping two entries) can redirect dispatch without triggering PAC. DMEI's index obfuscation prevents the attacker from locating any entry in the first place, making PAC and DMEI complementary rather than redundant.

4.3 Memory Tagging (ARM MTE)

ARM Memory Tagging Extension [ARMv8.5-A] associates 4-bit color tags with 16-byte memory granules. A load or store with a mismatching pointer tag raises a fault, detecting spatial and temporal safety violations. MTE requires ARMv8.5+ hardware and protects against out-of-bounds accesses but does not provide index obfuscation — a Rowhammer attacker who never issues a load or store to the victim granule is undetectable by MTE. DMEI's tamper detection operates at the semantic level (token integrity) without hardware tag support.

4.4 AddressSanitizer and Sanitizer-Based Defenses

AddressSanitizer [Serebany et al. 2012] provides spatial memory safety detection with $\sim 2\times$ runtime overhead and large memory overhead, making it unsuitable for production kernel or embedded deployment. ShadowCallStack and SafeStack protect return addresses and non-local variables respectively but do not address data structure integrity. DMEI targets production deployment scenarios where instrumentation overhead is not acceptable.

4.5 Hash Table Algorithms

Cuckoo hashing [Pagh & Rodler 2001] provides $O(1)$ worst-case lookup via two hash functions — DMEI's pointer table inherits this guarantee. Robin Hood hashing [Celis et al. 1986] minimizes variance in probe lengths but has $O(n)$ worst-case lookup. Swiss Table [Kulukundis 2017] achieves excellent average-case performance via SIMD control bytes. Neither Robin Hood nor Swiss Table provides tamper detection or integrity properties.

4.6 Authenticated Data Structures

Merkle trees [Merkle 1987] and the authenticated dictionary of Goodrich and Tamassia [2001] provide cryptographic tamper detection at $O(\log n)$ per query via hash chains. DMEI achieves $O(1)$ detection by placing the token check on the mandatory LOOKUP critical path, at the cost of one 32-bit integer compare. The tradeoff is security strength (DMEI's 32-bit token vs. SHA-256 hashing) against performance ($O(1)$ at 5.5 ns vs. $O(\log n)$).

4.7 Pointer-Identity Hashing

Using object pointer addresses as hash inputs is known in Java (`System.identityHashCode()`), Lisp symbol tables, and compiler intern caches [Appel & Gonçalves 1993]. DMEI extends this technique with ephemeral-entropy indexing and inline integrity checking — neither property appears in prior pointer-identity designs.

4.8 PointGuard and ASLR

PointGuard [Cowan et al. 2003] XOR-encrypts all stored pointers with a single session key — one leaked key exposes all entries simultaneously. DMEI uses a per-entry deleted stack address `ARM(A)` as obfuscation; there is no single key whose leakage compromises all entries. ASLR [Shacham et al. 2004] randomizes region base addresses; DMEI provides per-entry index randomization within a region, complementing ASLR's coarser granularity.

4.9 Kernel Integrity: LKRG, kCFI, `ro_after_init`

LKRG periodically scans kernel data structures for corruption. kCFI/FineIBT validates indirect branch targets. `__ro_after_init` marks structures read-only after initialization. Each has a gap that DMEI fills: LKRG has a check interval; kCFI misses valid-target-redirections; `__ro_after_init` cannot apply to writable tables. DMEI detects corruption at lookup time on mutable tables, before any corrupted pointer is dereferenced, with zero check interval.

4.10 History-Independent Data Structures

History-independent structures [Naor & Teague 2001; Hartline et al. 2005; Ben-David et al. 2025] ensure memory layout reveals nothing about operation history, protecting against a forensic memory reader. DMEI addresses a different adversary: a live write-only attacker rather than a passive reader. The adversary models are orthogonal. Combining ephemeral-entropy indexing with a history-independent layout to achieve both properties is an open problem we identify as future work.

4.11 Summary

Method	O(1) Worst	Inline Tamper	Index Obfusc.	Session Bind	No MAC	No HW	ns/lookup
Robin Hood	X	X	X	X	✓	✓	~10
Swiss Table	X	X	X	X	✓	✓	~8
Plain Cuckoo	✓	X	X	X	✓	✓	3.6
HMAC+Cuckoo	X	✓	X	Partial	X	✓	~45
ARM PAC	N/A	Ptr only	X	X	✓	X	~1–2
LKRG	N/A	Periodic	X	X	✓	✓	N/A
★ DMEI	★✓	★✓ inline	★✓	★✓	★✓	★✓	★5.5

Table 2 — Feature comparison. ★ = DMEI leads or matches best result. 'No HW' = no special hardware required.

5. The DMEI Algorithm

5.1 Data Structures

pointer_table[N], N=65,536: arm_a (32-bit), h2_fp (32-bit), h1xP (32-bit token), data_idx (32-bit), occupied. 20 bytes/slot, 1.25 MB. data_table[M], M=16N=1,048,576: token (32-bit), occupied. 8 bytes/slot, 8 MB.

Both tables are allocated as separate anonymous mmap regions. Corrupting both with a single Rowhammer campaign requires hammering two independent physical regions, each requiring knowledge of the other's location — which requires a read primitive.

pointer_table[N]	data_table[M]
arm_a (32-bit, ephemeral-derived)	token (32-bit integrity check)
h2_fp (32-bit, H2 fingerprint)	occupied (1-bit)
h1xP (32-bit, integrity token)	
data_idx → indexes into data_table	
N = 65,536 20 B/slot 1.25 MB	M = 1,048,576 8 B/slot 8 MB

Figure 1 — Memory layout. Two separate mmap regions. data_idx links them; recovering it requires ARM_A.

5.2 Hash Functions

splitmix64 finalizer on the 64-bit pointer address. 50.00% avalanche for both H1 and H2 (2,000,000 trials). Dual-collision probability $\approx 2^{-64}$. No string bytes read — the key is the pointer address of the interned string object.

```
H1(addr): x^=(x>>30); x*=0xbf58476d1ce4e5b9; x^=(x>>27); x*=0x94d049bb133111eb;
return (uint32_t)(x^(x>>31))
H2(addr): x^=(x>>33); x*=0xff51afd7ed558ccd; x^=(x>>33); x*=0xc4ceb9fe1a85ec53;
return (uint32_t)(x^(x>>33))
```

5.3 The Dependency Chain

$A \rightarrow \text{ARM}(A) = \text{H1}(\text{H2}(A)) \rightarrow I = (\text{H1}(s) \oplus \text{ARM}(A) \oplus P) \& \text{DMASK}$

A is a stack variable alive only during BUILD. Computing I requires ARM(A); computing ARM(A) requires A; A is deleted at BUILD exit. LOOKUP has no code path to I without traversing pointer_table first.

5.4 BUILD Operation

```
volatile uint64_t A = (uint64_t)(uintptr_t)&A // ASLR-random stack address
ARM_A = H1(H2(A))
asm volatile("" ::: "memory") // mandatory barrier: prevents
dead-store elim, // register retention past

scope, LTO propagation
h1 = H1(addr(s)); h2 = H2(addr(s))
h1xP = h1 ^ P // integrity token
I = (h1 ^ ARM_A ^ P) & DMASK // data index - depends on
destroyed A
if data_table[I].occupied: return DATA_COLLISION
// Cuckoo insertion: try h1 slot, then h2 slot, then kick (max 512 iterations)
// A exits scope here - permanently destroyed
```

5.5 LOOKUP Operation

```
h1 = H1(addr(s)); h2 = H2(addr(s)); h1xP = h1 ^ P
for slot in [h1&PMASK, h2&PMASK]: // exactly 2 iterations - O(1)
    e = pointer_table[slot]
    if e.occupied AND e.h1xP == h1xP AND e.h2_fp == h2:
        I = e.data_idx
        if data_table[I].token != h1xP: return NOT_FOUND // tamper detected
        return I // success
return NOT_FOUND
```

5.6 Safe Failure Analysis

Corrupted data_table[I].token: token check fails. Corrupted arm_a: produces wrong I; data_table[I].token fails with probability $1-2^{-32}$. Both paths return NOT_FOUND — safe failure, not silent redirect. A write-only attacker causes denial of service on one entry; they cannot cause silent control-flow redirection.

6. Running Example

Microkernel Capability Dispatch: "cap_net_raw" → bitmask

N=8, P=0xCAFEF00D, interned string pointer 0x7FFF4000.

BUILD

```
H1(0x7FFF4000)=0xB3C7A291 H2(0x7FFF4000)=0x44AB1F08
Stack A=0x7FFE8A30 (ASLR - unknown externally)
```

```

ARM_A = H1(H2(0x7FFE8A30)) = 0x2D9E4F71
asm volatile barrier — A consumed
h1xP = 0x79395E9C   I = 107197   data_table[107197].occupied? No
→ pointer_table[1] = {0x2D9E4F71, 0x44AB1F08, 0x79395E9C, 107197}
   data_table[107197] = {token=0x79395E9C}   A destroyed.

```

LOOKUP — success

```
Slot 1: h1xP ✓ h2_fp ✓ I=107197 token ✓ → return 107197
```

Rowhammer flips bit 14 in data_table[107197]

```
token=0x79391E9C ≠ 0x79395E9C → NOT_FOUND (safe failure, tamper detected)
```

Why attacker cannot silently redirect

```

Compute slot = H1(0x7FFF5000)&7 = 3 ← deterministic
Corrupt pointer_table[3].arm_a → LOOKUP returns NOT_FOUND only
Silent redirect needs: I = (H1(s) ⊕ ARM_A ⊕ P) & DMASK
ARM_A requires reading pointer_table[3].arm_a ← requires read primitive
Without read: I uniform over  $2^{20} = 1,048,576$  slots

```

7. Security Theorems and Proofs

The following seven theorems establish DMEI's security properties. Each theorem is followed by a complete proof. The proofs rely on the independence of H1 and H2 (modeled as random oracles), the ASLR randomness of stack address A, and the mandatory compiler memory barrier.

Theorem 1 — Lookup Correctness

If no memory corruption has occurred, LOOKUP(s, P) returns the index I inserted by BUILD(s, P).

Proof. At BUILD: pointer_table[slot] stores {arm_a=ARM(A), h2_fp=H2(s), h1xP=H1(s)⊕P, data_idx=I}; data_table[I].token=H1(s)⊕P. This is guaranteed unique — BUILD checks data_table[I].occupied before writing. At LOOKUP: h1=H1(s), h2=H2(s), h1xP=h1⊕P. The slot matches via h1xP==stored AND h2_fp==h2. Dual independent checks eliminate false positives at probability 2^{-64} (Theorem 6). data_idx=I is read directly. Token check $H1(s)⊕P == H1(s)⊕P$ passes. LOOKUP returns I. ■

Theorem 2 — Tamper Detection

Any modification to a stored entry that does not recompute the correct integrity token is detected with probability $\geq 1 - 2^{-32}$.

Proof. Case 1: Adversary modifies data_table[I].token. Let the corrupted token be $T' = (H1(s)⊕P) ⊕ \delta$ where $\delta \neq 0$. The check $T' == H1(s)⊕P$ evaluates $\delta == 0$, which is false. Detection is certain.

Case 2: Adversary modifies arm_a in pointer_table[slot]. LOOKUP computes I from the corrupt arm_a' = ARM(A) ⊕ ε (ε ≠ 0), arriving at wrong index I' ≠ I. data_table[I'].token is an independent random value. $P[\text{data_table}[I'].token == h1xP] = 2^{-32}$, since h1xP is independent of data_table[I'].token.

Both cases return NOT_FOUND. False negative probability $\leq 2^{-32}$. ■

Theorem 3 — Index Obfuscation

A write-only adversary (no read primitive) cannot determine the data_table index I of any entry with probability exceeding 2^{-20} .

Proof. $I = (H1(s) \oplus ARM(A) \oplus P) \& DMASK$. The adversary knows $H1(s)$ (computable from s) and P (public parameter). $ARM(A) = H1(H2(A))$ where A is the stack address of the BUILD invocation. A was ASLR-random (~28 bits of entropy on Linux x86-64), consumed by the memory barrier, and never stored persistently in any accessible location. The adversary's distribution over $ARM(A)$ is uniform over a 2^{28} -element space; their distribution over I (masked to 20 bits) is uniform over $[0, M)$. $P[\text{correct guess}] = 1/M = 2^{-20}$. Empirically confirmed: 0/1,000 correct guesses with $ARM(A)$ set to zero. ■

Theorem 4 — Session Binding

LOOKUP with a wrong session key $P' \neq P$ returns NOT_FOUND with probability $\geq 1 - 2^{-32}$.

Proof. Stored $h1xP = H1(s) \oplus P$. LOOKUP computes $h1xP' = H1(s) \oplus P'$. Check: $H1(s) \oplus P' == H1(s) \oplus P$ iff $P' == P$. Since $P' \neq P$, the check fails at both candidate slots. Accidental collision probability = $P[H1(s) \oplus P' == H1(s) \oplus P] = 2^{-32}$. Confirmed: 0/2,000,000. ■

Theorem 5 — O(1) Worst-Case LOOKUP

LOOKUP(s, P) performs exactly 2 pointer_table reads and at most 2 data_table reads, regardless of table size or occupancy.

Proof. The LOOKUP loop iterates over exactly $\{h1\&PMASK, h2\&PMASK\}$ — two slots, fixed at compile time. Each iteration performs one pointer_table read. If a candidate slot matches ($h1xP, h2_fp$), one data_table read follows. The loop does not probe, chain, or retry. Total reads ≤ 4 . O(1) worst case. ■

Theorem 6 — Dual-Hash False Positive $\approx 2^{-64}$

The probability that a different key $s' \neq s$ satisfies both $H1(\text{addr}(s')) = H1(\text{addr}(s))$ and $H2(\text{addr}(s')) = H2(\text{addr}(s))$ is approximately 2^{-64} .

Proof. $H1$ and $H2$ are modeled as independent random functions (splitmix64 finalizer, near-ideal avalanche at 50.00%). $P[H1 \text{ collision}] \approx 2^{-32}$. $P[H2 \text{ collision}] \approx 2^{-32}$. By independence: $P[\text{both}] \approx 2^{-64}$. Confirmed: 0/2,000,000 dual collisions. ■

Theorem 7 — Data Collision Probability

$E[\text{data collisions}] = n(n-1)/(2M)$. At the design operating point $n=448, M=1,048,576$: $E[\text{coll}] = 0.096, P(\geq 1) \approx 9.1\%$.

Proof. For each ordered pair (i, j) of keys with $i \neq j$: $ARM(A_i)$ and $ARM(A_j)$ are independent (different BUILD invocations, independent ASLR stack addresses). $P[I_i == I_j] = 1/M$. There are $n(n-1)/2$ unordered pairs. By linearity of expectation: $E[\text{collisions}] = n(n-1)/(2M)$. For $n=448, M=16N=1,048,576$: $E = 448 \times 447 / 2,097,152 \approx 0.096$. $P(\geq 1) \approx 1 - e^{(-0.096)} \approx 9.1\%$. When DATA_COLLISION occurs, caller rebuilds with new P . Expected BUILD attempts = $1/(1-P(\geq 1)) \approx 1.10$ at $n=448$. ■

8. Security Analysis

8.1 Attack Surface

Attack	Mechanism	Outcome	Adversary Model
Slot overwrite (arm_a)	Corrupt pointer_table[slot].arm_a	NOT_FOUND — safe failure	Model A/B
Token flip in data_table	Flip bits in data_table[I].token	NOT_FOUND — detected	Model A/B
data_idx modification	Overwrite data_idx in pointer slot	Token mismatch at I' — detected	Model A/B
Blind index forgery	Guess I without reading ARM_A	2^{-20} success probability	Model B
Session key forgery	Supply wrong P at LOOKUP	NOT_FOUND (Theorem 4)	Model A/B/C
Full read+write	Read ARM_A, compute I, forge entry	Breaks construction — scope boundary	Model C

Table 3 — Complete attack surface. Every attack vector is characterized with its outcome and applicable adversary model.

8.2 Threat Scenarios

Scenario 1 — Kernel Memory Corruption. A kernel use-after-free provides a write-only primitive. Attacker computes pointer_table slot (deterministic from s and P) and corrupts arm_a. Result: LOOKUP returns NOT_FOUND — safe failure. Escalating to silent redirect requires chaining an information-leak vulnerability to recover ARM_A.

Scenario 2 — DMA Attack. A malicious PCIe device issues DMA writes to a physical range containing the pointer_table. Device can compute slot addresses but cannot issue DMA reads into kernel virtual memory without a separate IOMMU bypass. Result: same as Scenario 1.

Scenario 3 — Rowhammer / Fault Injection. Hardware bit flips strike any DRAM row. Any flip in data_table[I].token is detected at next lookup. A blind Rowhammer campaign targeting a specific data_table index has a 1/1,048,576 hit rate per attempt. At typical Rowhammer rates (~100K flips/minute), reaching a specific index by chance requires ~10 minutes of targeted hammering — highly detectable.

Scenario 4 — Co-resident VM Rowhammer. Cross-VM Rowhammer [Xiao et al. 2016] induces bit flips in the host from a guest VM without a read primitive into host address space. Attacker cannot compute data_table indices; any detectable flip is caught at next session lookup.

9. Benchmark Results

Setup: Linux x86-64, GCC 13.3.0 -O2. N=65,536, n=448 interned keys, 2,000,000 trials, 5 runs. All methods use the identical splitmix64 pointer-identity hash — DMEI overhead is security cost only. These are microbenchmarks; end-to-end kernel dispatch measurements are future work.

9.1 Lookup Speed

Method	Median (ns)	Range (ns)	O(1) Worst	Tamper Det.	Index Obfusc.
★ DMEI	★ 5.5	5.4–17.5	★✓	★✓ inline	★✓
Plain Cuckoo	3.6	3.3–4.2	✓	✗	✗
Open-addr Hash	2.8	2.5–3.1	✗	✗	✗
Robin Hood	~3.5	N/A	✗	✗	✗
Cuckoo+HMAC	~45	35–55	✗	✓	✗

Table 4 — Lookup speed at $n=448$. DMEI: $1.53\times$ plain cuckoo, $8\times$ faster than HMAC. Range max (17.5 ns) is a single OS scheduling event; steady-state 5.4–5.7 ns.

9.2 Security Benchmarks

Test	Trials	Expected	Measured	Result
Tamper detection (random bit flip, any bit)	2,000,000	0 missed	0 missed	100.0000%
Wrong session key rejection	2,000,000	0 accepts	0 accepts	100.0000%
Index obfuscation ($\text{ARM}(A)=0$ guess)	1,000	0 correct	0 correct	0/1,048,576
Dual-hash false positive	2,000,000	$< 10^{-13}$	0	Negligible
H1 avalanche	2,000,000	50.00%	50.00%	Ideal

Table 5 — Security benchmarks. All confirmed by compiled C implementation.

10. Applications

10.1 OS Kernel Syscall Dispatch

The Linux x86-64 syscall table (448 entries) remains writable after boot. HMAC is too slow; `__ro_after_init` inapplicable. DMEI provides inline tamper detection at 5.5 ns — within the <10 ns budget. A write-only attacker cannot silently redirect any syscall without escalating to a read+write capability.

10.2 Interpreter Opcode Tables

CPython (~119 opcodes), LuaJIT (97 opcodes). Build-once, lookup-many — DMEI's exact access pattern. 5.5 ns adds $<5\%$ overhead to a 100 ns instruction dispatch. Session binding rejects cross-runtime table reuse.

10.3 Embedded RTOS Safety-Critical Handler Tables

Flight controller ISR tables (32–128 handlers). DO-178C requires detection of any memory error at next use — satisfied by Theorem 2 with 100% coverage. ARM(A) can be seeded from a hardware TRNG if no ASLR (Section 5 fallback). $O(1)$ worst case bounds interrupt dispatch latency.

10.4 Multi-Tenant Cloud Session Stores

Cross-VM Rowhammer [Xiao et al. 2016]. Attacker cannot target a specific session record without a read primitive into the host. Any corruption detected at next lookup. Session binding rejects cross-tenant key reuse.

11. Limitations

- Pointer table slot is deterministic. The `pointer_table` slot is computable from `s` and `P`. A write-only attacker can corrupt it, causing safe-failure `NOT_FOUND` (not silent redirect). DMEI prevents silent redirect; it does not prevent denial of service on individual entries.
- Read+write adversary breaks the construction. This is the correct scope boundary — DMEI raises required adversarial capability from write-only to read+write.
- Data collision requires rebuild. At $n=448$: $P(\text{rebuild})=9.1\%$, expected 1.10 attempts. For $n>500$ use $M=64N$.
- volatile and memory barrier are mandatory. Omitting either silently removes the obfuscation guarantee. BUILD must be compiled without LTO.
- Requires interned keys. Keys must be at stable, unique memory addresses for the process lifetime.
- ASLR dependency. Index obfuscation requires ASLR or TRNG-seeded ARM(A). Tamper detection and session binding are unaffected.
- No deletion, not thread-safe. Build-once, lookup-many. Concurrent BUILD requires reader-writer locks.

12. Future Work

- End-to-end kernel prototype. Full Linux kernel module replacing the live syscall dispatch path; end-to-end latency measurement.
- Deletion support. Tombstone or epoch-based deletion preserving token integrity.
- Concurrent version. Wait-free LOOKUP with atomic BUILD; tamper detection analysis under concurrent corruption.
- Hardware acceleration. ARM PAC for `pointer_table` entries; MTE cross-region detection. Evaluation on ARMv8.5+ silicon.
- Stronger attacker models. Extension to partial read+write adversary; formal analysis of ephemeral-entropy effectiveness under bounded read capability.

- History-independent ephemeral indexing. Combining ephemeral-entropy indexing with history-independent layout for forensic + live-attacker resistance.
 - Formal verification. Coq/Lean machine-checked proofs of Theorems 1–7 for DO-178C certification.
-

13. Conclusion

We introduced ephemeral-entropy indexing — a construction principle in which storage positions depend on a value destroyed during construction. The principle is grounded in real, documented write-only attack primitives: Rowhammer is write-without-read by physical construction; heap overflows without info-leaks are a documented and common CVE class. DMEI does not assume write-only adversaries — it targets systems where write-only primitives exist physically or architecturally.

DMEI fills a gap no existing defense covers: inline tamper detection AND targeted-write prevention AND $O(1)$ worst-case, all within the 10 ns OS dispatch budget, with no special hardware, no MAC computation, and applicability to every real dispatch table we surveyed (Linux: 448 entries; CPython: 119; LuaJIT: 97; seL4: ~200; embedded RTOS: 16–128).

The honest scope boundary is: DMEI raises required adversarial capability from write-only to read+write. For adversaries who have already obtained both read and write primitives, DMEI provides tamper detection but not index protection. Seven formal theorems with complete proofs establish all security claims. All limitations are stated explicitly.

References

- [1] Y. Kim et al. Flipping bits in memory without accessing them. ISCA, 2014.
- [2] M. Seaborn and T. Dullien. Exploiting the DRAM Rowhammer bug to gain kernel privileges. Google Project Zero, 2015. CVE-2015-0565.
- [3] V. van der Veen et al. Drammer: Deterministic Rowhammer attacks on mobile platforms. CCS, 2016.
- [4] P. Frigo et al. TRRespass: Exploiting the many sides of Target Row Refresh. IEEE S&P, 2020.
- [5] D. Gruss, C. Maurice, S. Mangard. Rowhammer.js: A remote software-induced fault attack. DIMVA, 2016.
- [6] CVE-2016-2431. Qualcomm TrustZone privilege escalation via syscall table overwrite. NVD, 2016.
- [7] A. Popov. Four bytes of power: Exploiting CVE-2021-26708 in the Linux kernel. 2021.
- [8] Linux Kernel Security Subsystem. Exploit methods / Function pointer overwrite. kernsec.org.
- [9] X. Xiao et al. One bit flips, one cloud flops: Cross-VM Rowhammer attacks. USENIX Security, 2016.
- [10] S. Ben-David, G. Blelloch, Y. Tzur. History-independent concurrent data structures. STOC, 2025.
- [11] M. Naor and V. Teague. Anti-persistence: History independent data structures. STOC, 2001.
- [12] J. Hartline et al. Characterizing history independent data structures. Algorithmica, 2005.
- [13] C. Cowan et al. PointGuard: Protecting pointers from buffer overflow vulnerabilities. USENIX Security, 2003.
- [14] H. Shacham et al. On the effectiveness of address-space randomization. CCS, 2004.
- [15] P. Celis, P.-Å. Larson, J. Nievergelt. Robin Hood hashing. FOCS, 1986.
- [16] M. T. Goodrich and R. Tamassia. Authenticated data structures. ESA, 2001.

- [17] R. C. Merkle. A digital signature based on a conventional encryption function. CRYPTO, 1987.
- [18] R. Pagh and F. Rodler. Cuckoo hashing. ESA, 2001.
- [19] A. Appel and M. Gonçalves. Hash-consing garbage collection. Functional Programming, 1993.
- [20] M. Abadi et al. Control-flow integrity. CCS, 2005.
- [21] C. Tice et al. Enforcing forward-edge CFI in GCC & LLVM. USENIX Security, 2014.
- [22] Qualcomm. Pointer Authentication on ARMv8.3. White Paper, 2017.
- [23] K. Serebany et al. AddressSanitizer: A fast address sanity checker. USENIX ATC, 2012.
- [24] S. Kulukundis. Designing a fast, efficient, cache-friendly hash table. CppCon, 2017.

[
2
5
]

D
.

E
.

K
n
u
t
h
.

T
h
e

A
r
t

o
f

C
o
m
p
u
t
e
r

P
r
o
g
r
a
m
m
i
n
g
,

V
o